

End-to-End Agentic AI QA Workflow Prompt

Using AI Agents, MCP & Playwright | Autonomous QA Engineer

📁 Workflow Overview

This prompt guides you through a **complete 7-step QA workflow** using MCP servers and AI agents to go from user story to committed automated test scripts. Each step builds on the output of the previous one, creating a fully traceable QA pipeline.

Tech Stack:

- **AI Agent:** Claude (via MCP)
 - **Browser Automation:** Playwright MCP browser tools
 - **Test Framework:** Playwright (JavaScript)
 - **Version Control:** GitHub MCP agent
 - **Target App:** SauceDemo eCommerce (or your application)
-

📁 Project Folder Structure

```
AgenticAIQAE2E_Workflow/  
├── user-stories/  
│   └── SCRUM-101-ecommerce-checkout.md  
├── specs/  
│   └── saucedemo-checkout-test-plan.md  
├── tests/  
│   └── saucedemo-checkout/  
│       ├── checkout-happy-path.spec.ts  
│       ├── checkout-validation.spec.ts  
│       └── checkout-navigation.spec.ts  
├── test-results/  
│   └── SCRUM-101-checkout-test-report.md  
├── playwright.config.ts  
├── package.json  
└── mcp.json
```

✅ STEP 1: Read User Story

Agent: Default (File Reader)

Prompt:

I need to start a new testing workflow. Please read the user story from the file:
user-stories/SCRUM-101-ecommerce-checkout.md

Summarize the key requirements, acceptance criteria, and testing scope.

Expected Output:

- Summary of the user story
 - List of acceptance criteria
 - Application URL and test credentials
 - Key features to test
-

📋 STEP 2: Create Test Plan

Agent: playwright-test-planner

Prompt:

Based on the user story SCRUM-101 that we just reviewed, use the playwright-test-planner agent to:

1. Read the application URL and test credentials from the user story
2. Explore the application and understand all workflows mentioned in the acceptance criteria
3. Create a comprehensive test plan that covers all acceptance criteria including:
 - Happy path scenarios
 - Negative scenarios (validation errors, empty fields, invalid data)
 - Edge cases and boundary conditions
 - Navigation flow tests
 - UI element validation
4. Save the test plan as: specs/saucedemo-checkout-test-plan.md

Ensure each test scenario includes:

- Clear test case title
- Detailed step-by-step instructions
- Expected results for each step
- Test data requirements

Expected Output:

- Complete test plan markdown file saved to `specs/`
- Organized test scenarios with clear structure
- Browser exploration screenshots (if needed)

STEP 3: Perform Exploratory Testing

Agent: Playwright MCP Browser Tools

Prompt:

Now I need to perform manual exploratory testing using Playwright MCP browser tools.

Please read the test plan from: specs/saucedemo-checkout-test-plan.md

Then execute the test scenarios defined in that plan:

1. Use Playwright browser tools to manually execute each test scenario from the plan
2. Follow the step-by-step instructions in each test case
3. Verify expected results match actual results
4. Take screenshots at key steps and error states
5. Document your findings:
 - Test execution results for each scenario
 - Any UI inconsistencies or unexpected behaviors
 - Missing validations or bugs discovered
 - Screenshots as evidence

Expected Output:

- Manual test execution results
- Screenshots of the application at various states
- List of observations and findings
- Any issues discovered during exploration

STEP 4: Generate Automation Scripts

Agent: `playwright-test-generator`

Prompt:

Now I need to create automated test scripts using the playwright-test-generator agent.

Please review:

1. Test plan from: specs/saucedemo-checkout-test-plan.md
(for test scenarios and steps)
2. Exploratory testing results from Step 3
(for actual element selectors and UI insights)

Using insights from the manual exploratory testing:

- Leverage the element selectors and locators that were successfully used in Step 3
- Use stable element properties (IDs, data attributes, roles) discovered during exploration
- Apply wait strategies and UI behaviors observed during manual testing
- Incorporate any workarounds for UI quirks discovered

Generate Playwright JavaScript automation scripts:

1. Create scripts for each test scenario from the test plan
2. Organize scripts into appropriate test suite files in: tests/saucedemo-checkout/
3. Use the test case names and steps from the test plan
4. Use reliable selectors and strategies from exploratory testing

Requirements for all scripts:

- Follow Playwright best practices
- Include proper assertions using expect()
- Use descriptive test names matching the format in the test plan
- Use robust selectors discovered during manual testing
- Add comments for complex steps
- Use proper wait strategies based on actual application behavior
- Add proper test hooks (beforeEach, afterEach)
- Configure for multiple browsers (Chrome, Firefox, Safari)

After generating the scripts, run the tests to verify they pass.

Expected Output:

- All scripts follow Playwright best practices
- Initial test generation complete
- Scripts saved in tests/saucedemo-checkout/
- Test run initiated

STEP 5: Execute and Heal Automation Tests

Agent: playwright-test-healer

Prompt:

Now I need to execute the generated automation scripts and heal any failures using the playwright-test-healer agent.

1. Run all automation scripts in: tests/saucedemo-checkout/
2. Identify any failing tests
3. For each failing test, use the playwright-test-healer agent to:
 - Analyze the failure (selector issues, timing issues, assertion failures)
 - Auto-heal the test by fixing selectors, adding waits, or adjusting assertions
 - Update the test script with the fixes
4. Re-run the healed tests to verify they pass
5. Repeat the heal process until all tests are stable and passing
6. Document:
 - Initial test results (pass/fail count)
 - Healing activities performed
 - Final test results after healing
 - Any tests that couldn't be auto-healed

Expected Output:

- All automation tests executed
- Failing tests identified and healed using test-healer agent
- Healed test scripts updated in tests/saucedemo-checkout/
- Final stable test execution results
- Summary of healing activities performed

STEP 6: Create Test Report

Agent: Default (File Writer)

Prompt:

Now I need to create a comprehensive test execution report based on manual testing, automation execution, and healing activities.

Please compile results from:

- Step 3: Manual exploratory testing results
- Step 4: Generated automation scripts
- Step 5: Automated test execution and healing results

Structure the report as: test-results/SCRUM-101-checkout-test-report.md

Include:

1. Executive Summary

- Total test cases planned
- Test cases executed (manual + automated)
- Overall Pass/Fail/Blocked status

2. Manual Test Results

- Results from Step 3 exploratory testing
- Screenshots and observations
- Issues found during manual testing

3. Automated Test Results

- Initial automation results from Step 5
- Healing activities performed
- Final test execution results after healing
- Test suite execution summary
- Pass/Fail count for each test suite

4. Defects Log

For any failed tests (manual or automated):

- Bug ID
- Severity (Critical / High / Medium / Low)
- Title and Description
- Steps to Reproduce
- Expected vs Actual Behavior
- Screenshots/Evidence
- Environment Details

5. Test Coverage Analysis

- Which acceptance criteria are covered
- Coverage from manual vs automated tests

- Any gaps in test coverage
- Recommendations for additional testing

6. Summary and Recommendations

- Overall quality assessment
- Risk areas
- Next steps

Expected Output:

- Comprehensive test execution report covering both manual and automated testing
 - Clear PASS/FAIL status for all test scenarios
 - Detailed bug reports for failures
 - Complete test coverage analysis
 - Evidence and screenshots attached
-

STEP 7: Commit to Git Repository

Agent: GitHub MCP Agent

Git Repository URL:

```
https://github.com/<your-username>/AgentE2EQAWorkflow-Playwright.git
```

Prompt:

Now I need to commit all the test artifacts to the Git repository using the GitHub MCP agent.

Git Repository URL: `https://github.com/<your-username>/AgentE2EQAWorkflow-Playwright.git`

Please perform the following Git operations:

1. Initialize Git repository if not already initialized
2. Stage all files in the workspace (all new and modified files)
3. Create a commit with the message:
"feat(tests): Add complete test suite for SCRUM-101 checkout workflow"

Including:

- Add user story documentation
- Add comprehensive test plan with all scenarios
- Add test execution report with results
- Add automated test scripts for checkout process
- Include validation, navigation, and edge case tests
- Resolves SCRUM-101

4. Push all changes to the Git repository
5. Provide a summary of what was committed

Expected Output:

- All workspace files committed to Git
- Descriptive commit message following conventional commit format
- Confirmation of successful push to the provided repository
- Summary of changes

Complete Workflow Execution — Single Combined Prompt (For Video Demo)

Use this single master prompt to trigger the **entire 7-step workflow at once** in one agent session:

STEP 1 - READ USER STORY:

Read and summarize the user story from: `user-stories/SCRUM-101-ecommerce-checkout.md`

The agent should understand the key requirements, acceptance criteria, and testing scope.

STEP 2 - CREATE TEST PLAN:

Use the `playwright-test-planner` agent to create a comprehensive test plan based on the

user story. The agent should explore the application URL from the user story and cover

all acceptance criteria. Save it as: `specs/saucedemo-checkout-test-plan.md`

STEP 3 - EXPLORATORY TESTING:

Read the test plan from `specs/saucedemo-checkout-test-plan.md` and use Playwright browser

tools to manually execute each test scenario and note any issues discovered.

Document

findings with screenshots and notes.

STEP 4 - GENERATE AUTOMATION SCRIPTS:

Review both the test plan (`specs/saucedemo-checkout-test-plan.md`) and exploratory testing

results from Step 3. Use the `playwright-test-generator` agent to create JavaScript automation

scripts leveraging the element selectors and insights discovered during manual testing.

Save scripts in `tests/saucedemo-checkout/`.

STEP 5 - EXECUTE AND HEAL TESTS:

Run all automation scripts from `tests/saucedemo-checkout/`. Use the `playwright-test-healer`

agent to identify and auto-heal any failing tests. Re-run tests until all are stable and

passing. Document healing activities.

STEP 6 - CREATE TEST REPORT:

Create a comprehensive test execution report at:

`test-results/SCRUM-101-checkout-test-report.md`

Compile results from Step 3 (manual testing), Step 4 (script generation), and Step 5

(execution and healing). Include PASS/FAIL status, healing summary, defects log, and

test coverage analysis.

STEP 7 - COMMIT TO GIT:

Use the GitHub MCP agent to commit all new files with a descriptive message and push to the repository.

Execute this complete workflow and provide status updates after each step.

MCP Server Configuration (`mcp.json`)

```
json
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    },
    "playwright-test-planner": {
      "command": "node",
      "args": [". /agents/test-planner/index.js"]
    },
    "playwright-test-generator": {
      "command": "node",
      "args": [". /agents/test-generator/index.js"]
    },
    "playwright-test-healer": {
      "command": "node",
      "args": [". /agents/test-healer/index.js"]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "<YOUR_GITHUB_PAT>"
      }
    }
  }
}
```

Notes

- Replace `<your-username>` in the Git repository URL with your actual GitHub username.
- Replace `<YOUR_GITHUB_PAT>` with a valid GitHub Personal Access Token that has `repo` scope.
- The agents (`playwright-test-planner`, `playwright-test-generator`, `playwright-test-healer`) are custom MCP server agents built as part of this workflow project.
- Adjust the file paths (`user-stories/`, `specs/`, `tests/`, `test-results/`) to match your actual project structure.
- This workflow was demonstrated using the **SauceDemo** eCommerce checkout flow as the target application.

Reference: End-to-End Agentic AI QA Workflow with AI Agents, MCP & Playwright — Ravikanth FicusRoot YouTube: <https://www.youtube.com/watch?v=rwR1h8OGoaI>